



APLICACIONES MÓVILES

DOCENTE: GABRIEL EDUARDO MOREJÓN
LÓPEZ

ACTIVIDAD 2: AUTÓNOMA 1

UNIDAD 1: ARQUITECTURA DE APLICACIONES

NOMBRES Y APELLIDOS DEL ESTUDIANTE

MICHAEL AGUSTIN PALMA NAVARRETE
BLAIR STEVEN DELGADO CORNEJO
CHRITOFER JESUS BARZOLA INDARTE
LEIBER VALENTIN ZAMBRANO CHAVEZ

28 DE MAY DE 2026

Repositorio: https://github.com/RTgo14/Sistema_HelpDesk

Zip Drive:

<https://drive.google.com/drive/folders/1PzRqio08ysHNptC5I4Mez6QqCIQXhHR7?usp=sharing>

UNIDAD 1: ARQUITECTURA DE APLICACIONES

El presente informe técnico corresponde a la Actividad 2, enfocada en la refactorización y modularización del prototipo desarrollado previamente en JavaScript durante la Actividad 1. El sistema desarrollado por el Grupo 8 consiste en una mesa de ayuda para la gestión de incidencias tecnológicas, permitiendo registrar, visualizar y consultar problemas relacionados con soporte técnico.

El objetivo principal de esta fase fue reorganizar el código para mejorar su estructura interna, separando responsabilidades en diferentes módulos y aplicando validaciones reutilizables, reglas de negocio y mejoras visuales para optimizar la experiencia del usuario. Además, se buscó dejar una base técnica más mantenible y preparada para futuras integraciones.



Figura 1. Vista general del proyecto en VS Code mostrando la estructura de carpetas.

1. DESARROLLO

1.1 Objetivo del módulo:

El módulo de Mesa de Ayuda e Incidencias Tecnológicas tiene como propósito facilitar el registro y consulta de problemas tecnológicos mediante una interfaz web interactiva.

El sistema permite al usuario registrar incidencias indicando un título, descripción y nivel de prioridad. Posteriormente, estas incidencias pueden ser consultadas, filtradas y visualizadas detalladamente mediante un modal interactivo. Como parte de la refactorización, se implementó una organización modular del código para mejorar la mantenibilidad del sistema, reduciendo la duplicación de lógica y permitiendo reutilizar validaciones y funciones importantes dentro del proyecto.

1.2 Descripción del flujo principal del usuario

Al entrar al panel principal, lo primero que te recibe es la lista de reportes organizados de los más recientes a los más antiguos. Si necesitas registrar un problema nuevo, solo debes llenar un formulario rápido; el sistema se encarga de revisar que todo esté en orden y, una vez guardado, no tienes que preocuparte si cierras el navegador, ya que la información se almacena en tu equipo de forma automática. Además, cuando la lista crezca, puedes usar la barra de búsqueda o el filtro de prioridad para encontrar cualquier reporte en segundos. Finalmente, si necesitas profundizar en algún ticket, basta con hacer clic en “Ver Detalle” para que se abra una ventana con toda la información completa.

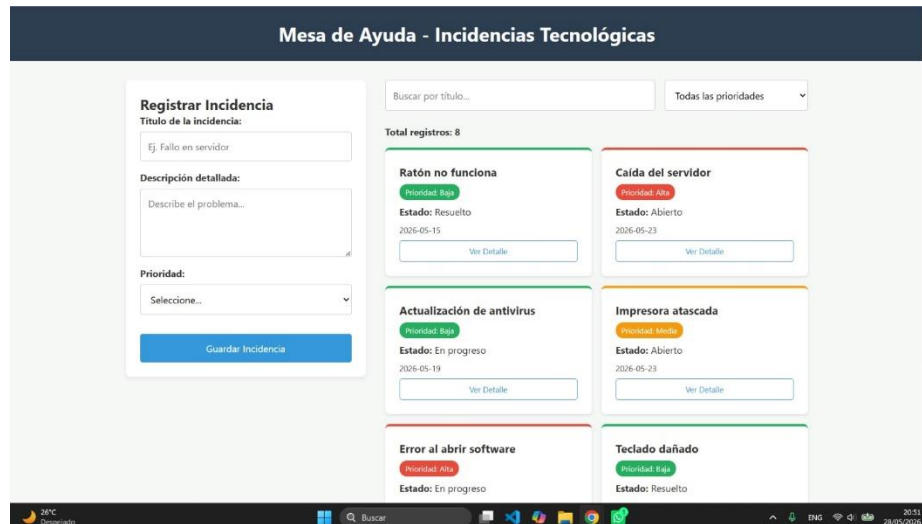


Figura 2. Dashboard principal funcionando

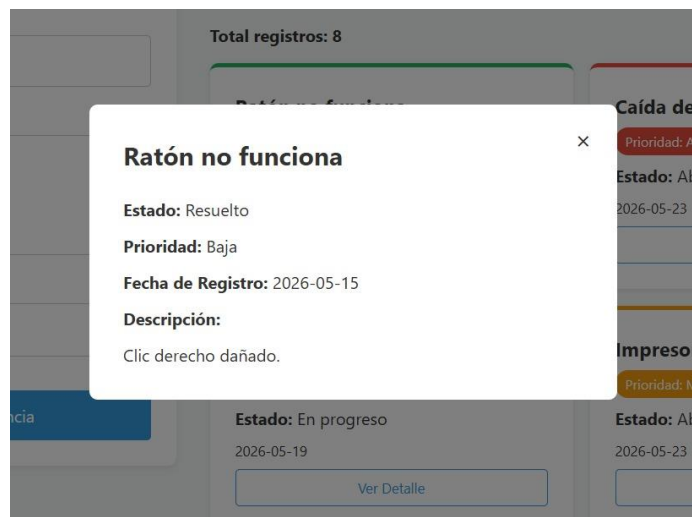


Figura 3. Modal abierto mostrando detalle de una incidencia

1.3 Explicación de la estructura de carpetas y archivos:

Para esta actividad se reorganizó el proyecto utilizando una estructura modular basada en JavaScript ES6 Modules, separando las responsabilidades principales del sistema.

index.html

Contiene la estructura visual del dashboard, incluyendo el formulario de registro, filtros de búsqueda, listado de incidencias y modal de detalle.

styles.css

Se encarga del diseño visual del sistema utilizando Flexbox, Grid y variables CSS para mantener una interfaz organizada y adaptable.

js/main.js

Actúa como el archivo principal encargado de conectar eventos del DOM con las funciones del sistema.

js/data/seed.js

Contiene los datos iniciales de prueba del sistema, permitiendo cargar incidencias predeterminadas.

js/services/storage.js

Centraliza la lógica de persistencia mediante localStorage, evitando repetir código relacionado con el almacenamiento de datos.

js/services/module-service.js

Gestiona la lógica principal del módulo, incluyendo operaciones relacionadas con incidencias, filtros y manipulación de registros.

js/ui/render.js

Se encarga exclusivamente del renderizado visual de las incidencias dentro del DOM.

js/utils/validators.js

Incluye funciones reutilizables para validar entradas del formulario, evitando registros inválidos.

js/models/entity.js

Contiene la estructura principal de la entidad Incidencia, permitiendo estandarizar los objetos creados dentro del sistema.

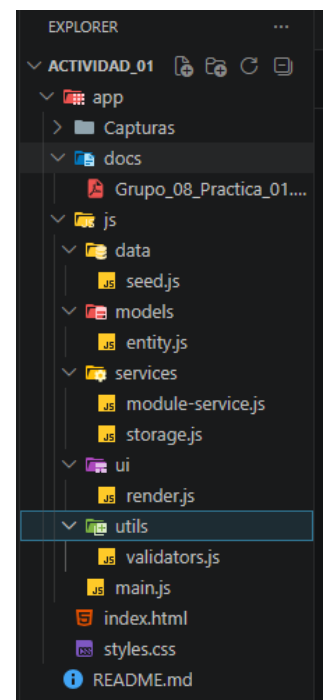


Figura 4. Estructura de carpetas

1.4 Funciones principales y reglas de negocio implementadas:

Durante la refactorización se implementaron varias funciones importantes para garantizar el correcto funcionamiento del sistema.

Renderizado dinámico de incidencias

El sistema utiliza el método `map()` para generar automáticamente las tarjetas de incidencias dentro del DOM, evitando crear manualmente cada elemento HTML.

```
const htmlString = datosOrdenados.map(incidencia => `

Figura 5. Uso de método map()



Además, mediante sort() se ordenan los registros por fecha o identificador para mostrar primero las incidencias más recientes.



```
const datosOrdenados = [...datos].sort((a, b) => b.id - a.id);
```



Figura 6. Método sort()



### Visualización detallada



Se implementó un modal interactivo utilizando el método find(), el cual permite localizar una incidencia específica mediante su identificador y mostrar sus datos completos.



```
const id = Number(e.target.dataset.id);
const incidencia = incidenciasActuales.find(inc => inc.id === id);
```



Figura 7. Método find()



## 1.5 Dificultades encontradas y soluciones



Durante el desarrollo nos topamos con algunos retos, especialmente al organizar el código y manejar los elementos dinámicos. Uno de los mayores dolores de cabeza fue que el botón de “Ver Detalle” dejaba de funcionar cada vez que se actualizaba la lista de reportes; por suerte, lo resolvimos asegurándonos de volver a conectar esos eventos justo después de cargar las tarjetas en pantalla.



```
// Eventos Delegados (Mejora de rendimiento para el modal)
listaDOM.addEventListener('click', (e) => {
 if (e.target.dataset.action === 'detalle') {
 const id = Number(e.target.dataset.id);
 const incidencia = incidenciasActuales.find(inc => inc.id === id);

 if (incidencia) {
 document.getElementById('modal-titulo').textContent = incidencia.titulo;
 document.getElementById('modal-estado').textContent = incidencia.estado;
 document.getElementById('modal-prioridad').textContent = incidencia.prioridad;
 document.getElementById('modal-fecha').textContent = incidencia.fecha;
 document.getElementById('modal-descripcion').textContent = incidencia.descripcion;
 modal.classList.remove('hidden');
 }
 }
});
```



Figura 8. Código del Modal de detalles



4


```

También tuvimos un pequeño tropiezo técnico con los módulos de JavaScript, ya que el navegador bloqueaba la página si la abríamos de forma tradicional (con doble clic), pero lo solucionamos rápidamente levantando un servidor local con la extensión Live Server de Visual Studio Code. Al final, la clave fue hacer pruebas constantes después de cada ajuste, asegurándonos de que los filtros, el almacenamiento, el modal y las validaciones siguieran funcionando a la perfección en todo momento.

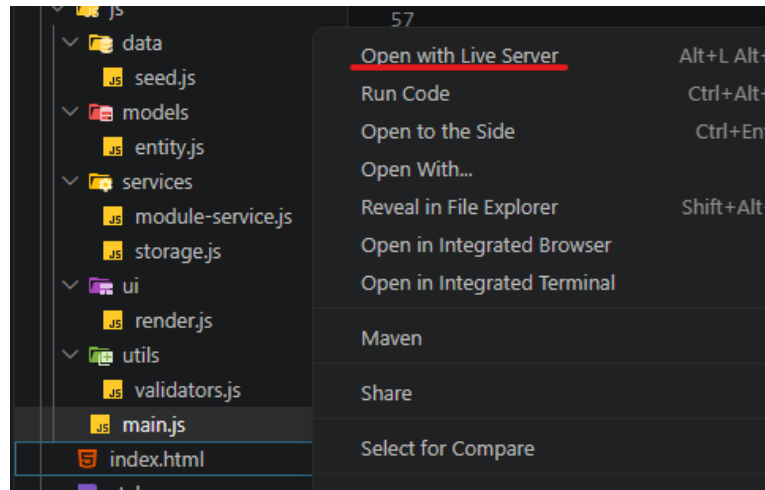


Figura 9. Uso de Live Server

1.6 Mejoras propuestas para fases posteriores

Pensando en cómo seguir haciendo crecer la plataforma, tenemos varias ideas en mente para las próximas etapas. El paso más importante será conectar el sistema a una base de datos real en lugar de depender del almacenamiento local, para que varios usuarios puedan trabajar al mismo tiempo y tener la información siempre sincronizada.

También queremos implementar un sistema de inicio de sesión que nos permita separar los accesos entre técnicos y administradores. Así mismo, otra mejora muy práctica será poder actualizar el avance de los reportes directamente desde la ventana de detalles, permitiendo cambiar un ticket de “Abierto” a “En progreso” o “Resuelto” sobre la marcha.

Finalmente, nos encantaría desarrollar una versión adaptada para móviles, así será mucho más fácil y rápido reportar cualquier eventualidad directamente desde el celular.

2. CONCLUSIONES

Concluimos que, en el transcurso de esta actividad se amplió el conocimiento de forma significativa para organizar y separar funcionalidades en el mismo sistema de Incidencias Tecnológicas HelpDesk. Donde la refactorización y modularización del proyecto ayudaron a separar las responsabilidades dentro del código, lo que facilita el correcto mantenimiento, reutilización y escalabilidad para futuras mejoras.

Y a su vez, se implementaron validaciones, filtros dinámicos, almacenamiento local y componentes interactivos como el modal de detalle, logrando que la experiencia para el usuario sea más fluida y ordenada. Durante el proceso también se presentaron dificultades técnicas relacionadas con el manejo de DOM y los módulos de JS, las cuales fueron solucionadas mediante pruebas constantes y una mejor estructura de evento.